iacc instituto profesional

EVALUACIÓN

Estructura de datos.
Semana 2

Amparo Valdivia
19, abril de 2025
Técnico en Análisis y programación

DESARROLLO:

La empresa “La Casa de SAM” es una tienda de productos electrónicos que se dedica a la venta de televisores, smartphones, laptops y otros productos tecnológicos. Actualmente la empresa maneja un inventario de 50 productos, cada uno con las siguientes características:

- Marca.
- Modelo.
- Tamaño de pantalla.
- Capacidad de almacenamiento.
- Procesador.
- Cámara.
- RAM.
- Unidades disponibles.
- Precio.

- Los diccionarios en Python almacenan datos como pares de clave: valor, se pueden agregar, eliminar y modificar elementos después de su creación, lo que permitiría en el inventario poder agregar un modelo nuevo, o modificar los datos de algún televisor que este descontinuado.
- Se accede a los valores utilizando sus claves en lugar de índices numéricos. **Ejemplo 1 televisores**,

- ```
  televisores= {'marca':['samsung', 'kioto', 'phillips', 'lg', 'sony', 'rec', 'aoc'],
  'precio':[150.000,200.000,250.000,300.000,350.000],'pantalla':[32,48,50,55],'modelo':['x10','x20','x30']}#diccionario, clave valor.
  ```
  ```

```
# diccionario, con datos ordenados por asociación.

televisores_datos = [
    {'marca': 'samsung', 'precio': 150000, 'pantalla': 32, 'modelo': 'x10'},
    {'marca': 'kioto', 'precio': 200000, 'pantalla': 48, 'modelo': 'x20'},
    {'marca': 'phillips', 'precio': 250000, 'pantalla': 50, 'modelo': 'x30'},
    {'marca': 'lg', 'precio': 300000, 'pantalla': 55},
    {'marca': 'sony', 'precio': 350000},
    {'marca': 'rec',},
    {'marca': 'aoc',}
]
#para ordenar,
def  sort_by_marca(televisor):#función para ordenar sort por marca parámetro televisor.
    return televisor['marca']
televisores_datos.sort(key=sort_by_marca)#si quiero que ordene de la a/z se deja así por default, sino (key=sort_by_marca,reverse=True)

#php para recorrer los arreglos tiene foreach, python for.

for televisor in televisores_datos:# para acceder al diccionario y solicitar tv. sony
    if televisor['marca'] == 'sony':
        print(televisor)
        # para el precio de Sony:
        if 'precio' in televisor:
            print(f"Precio de Sony: {televisor['precio']}")
            #para agregar a este diccionario una clave de stock:

televisores_datos = [
    {'marca': 'samsung', 'precio': 150000, 'pantalla': 32, 'modelo': 'x10', 'stock' :90},
    {'marca': 'kioto', 'precio': 200000, 'pantalla': 48, 'modelo': 'x20' , 'stock' :10},
    {'marca': 'phillips', 'precio': 250000, 'pantalla': 50, 'modelo': 'x30' , 'stock' :5},
    {'marca': 'lg', 'precio': 300000, 'pantalla': 55 , 'stock':40},
    {'marca': 'sony', 'precio': 350000, 'stock':12},
    {'marca': 'rec','precio':120000, 'stock':0},
    {'marca': 'aoc','precio':230000, 'stock':0}
]
```

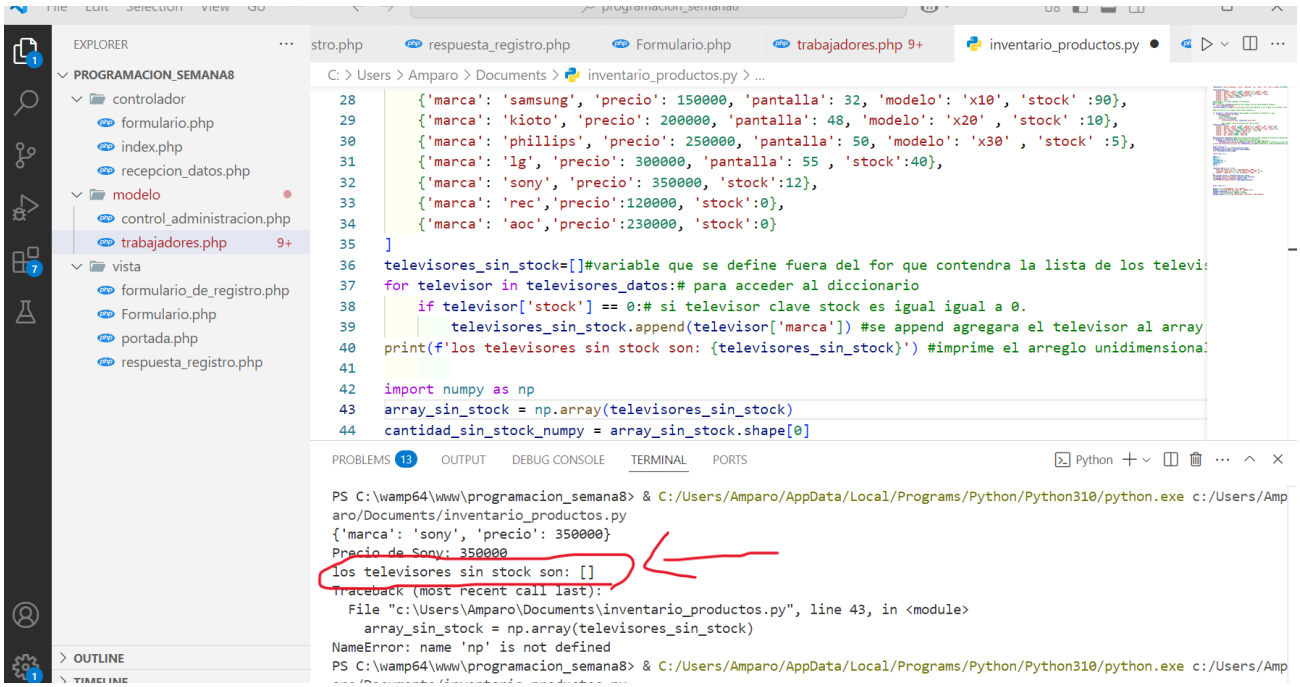
- Numpy es una librería de Python, que da soporte y herramientas matemáticas, para manejar datos, para su uso en este lenguaje importar como …

import numpy as np

- Un arreglo unidimensional se utilizaría para almacenar una única característica clave de cada producto en el inventario, por ejemplo, los televisores sin stock de inventario.

```
• televisores_sin_stock=[]#variable que se define fuera del for que contendrá
  la lista de los televisores sin stock.
• for televisor in televisores_datos:# para que recorra el arreglo.
•     if televisor['stock'] == '0':# si televisor clave stock es igual igual a
  0.
```

Error corregido no imprimió nada porque las comillas hacen que sea un string y no número y faltó import numpy as np.




```
• televisores_sin_stock=[]#variable que se define fuera del for que contendrá
  la lista de los televisores sin stock.
• for televisor in televisores_datos:# para acceder al diccionario
•     if televisor['stock'] == 0:# si televisor clave stock es igual igual a 0.
•
```

- `televisores_sin_stock.append(televisor[“marca”]) #se append agregará la marca al array televisores_sin_stock.`
- `print(f'los televisores sin stock son: {televisores_sin_stock}') #imprime el arreglo unidimensional que guardara los televisores sin stock.`

```
PROBLEMS 13   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                    Python

RAM del segundo celular: 8
PS C:\wamp64\www\programacion_semana8> & C:/Users/Amparo/AppData/Local/Programs/Python/Python310/python.exe c:/Users/Amp
aro/Documents/inventario_productos.py
{'marca': 'sony', 'precio': 350000}
Precio de Sony: 350000
los televisores sin stock son: [{'marca': 'rec', 'precio': 120000, 'stock': 0}, {'marca': 'aoc', 'precio': 230000, 'stoc
k': 0}]
2
```

- Los datos del inventario pueden almacenarse en listas y cuando los datos son grandes, pueden crearse sublistas, para guardar datos históricos. Una Tupla en el caso del inventario puede ser una lista unidimensional inmutable podría usarse para tener los datos de los códigos de los productos, que no cambian, por ejemplo, también se pueden poner los precios en un arreglo unidimensional para calcular el promedio, ordenar los datos y buscarlos.

Si paso una lista de Python como argumento np.array(), ejemplo: `televisores_sin_stock=[“rec”,”aoc”]`

- `array_sin_stock = np.array(televisores_sin_stock)# crea un array de la lista.`
- `cantidad_sin_stock_numpy = array_sin_stock.shape[0]#contando los elementos del array.`
- `print (cantidad_sin_stock_numpy)`

- Para poder utilizar una matriz con un arreglo bidimensional con numpy, primer ejemplo:

```
import numpy as np

MARCA = 0   # constantes que se escriben con mayúsculas índices de las columnas, facilita encontrar y llamar la posición que se necesita.
MODELO = 1
PANTALLA = 2
ALMACENAMIENTO = 3
PROCESADOR = 4
CAMARA = 5
RAM = 6

inventario_matriz = np.array([
    ["LG", "G8 ThinQ", 6.1, 128, "Snapdragon 855", "12MP...", 6],
    ["Samsung", "Galaxy S21", 6.2, 128, "Exynos 2100", "12MP...", 8],
    ["Apple", "iPhone 13", 6.1, 128, "A15 Bionic", "12MP...", 4]
])
```

- Segundo ejemplo, con listas unidimensionales:

```
marcas = np.array(["Samsung", "LG", "Sony"])
modelos = np.array(["QLED TV", "OLED TV", "BRAVIA TV"])
unidades_disponibles = np.array([10, 5, 12])
precios = np.array([1200.00, 1500.00, 1100.00])
codigos_producto = np.array(["SAM-TV-001", "LG-TV-002", "SON-TV-003"])

```

- La clave para implementar arreglos unidimensionales de manera efectiva para el control de inventario es mantener la correspondencia por índice en el ejemplo, para varios arreglos unidimensionales (uno para cada atributo), el elemento en la misma posición (mismo índice) en cada arreglo debe corresponder al mismo producto.

- NumPy permite realizar operaciones en lote sobre los datos (por ejemplo, aumentar todos los precios en un porcentaje), se puede acceder iterando por

la matriz en orden filas y columnas, donde cada fila representa un producto individual y cada columna representa un atributo específico del producto. Numpy lo hace, sin necesidad de realizar el ciclo for.

Solo Numpy

Matriz 3x3

```
Nombre_matriz[fila,columna]
Nombre_matriz[2,2]
Nombre_matriz[2,2]=1000
Print(Nombre_matriz)
Print(Nombre_matriz[:,0])=lg,aoc,ki
Print(Nombre_matriz[1,:])=aoc,$,num
```

Para acceder a la marca:

- `marca_primer_celular = inventario_matriz[0, MARCA]`
- `print(f"Marca del primer celular: {marca_primer_celular}")`

- `ram_segundo_celular = inventario_matriz[1, RAM]`
- `print(f"RAM del segundo celular: {ram_segundo_celular}")`

```
Marca del primer celular: LG
RAM del segundo celular: 8
```

- Agregé en la columna 16 un print para ver el arreglo ordenado y realizar operaciones como encontrar el precio promedio de todos los televisores

```
15 televisores_datos.sort(key=sort_by_marca)#si quiero que ordene de la a/z se deja asi por default,
16 print(televisores_datos)
```

-

Arreglo ordenado.

```
aro/Documents/inventario_productos.py
[{'marca': 'aoc'}, {'marca': 'kioto', 'precio': 200000, 'pantalla': 48, 'modelo': 'x20'}, {'marca': 'lg', 'precio': 3000
00, 'pantalla': 55}, {'marca': 'phillips', 'precio': 250000, 'pantalla': 50, 'modelo': 'x30'}, {'marca': 'rec'}, {'marca
': 'samsung', 'precio': 150000, 'pantalla': 32, 'modelo': 'x10'}, {'marca': 'sony', 'precio': 350000}]
{'marca': 'sony', 'precio': 350000}
Precio de Sony: 350000
los televisores sin stock son: ['rec', 'aoc']
2
Marca del primer celular: LG
RAM del segundo celular: 8
```

Operación promedio:

```
precio_televisores=[]#arreglo creado para guardar los precios.
for televisor in televisores_datos:
    precio_televisores.append(televisor['precio']) #agrega siempre precio.
print(f'los precios son: {precio_televisores}')# imprime precio.
# Calcular el precio promedio
precio_promedio_tv = np.mean(precio_televisores)
print(f"Precio promedio de los televisores: ${precio_promedio_tv:.2f}")
```

Resultado.

```
los precios son: [150000, 200000, 250000, 300000, 350000, 120000, 230000]
Precio promedio de los televisores: $228571.43
```

- Como lo indica Mancilla Herrera, en Diseño y construcción de algoritmos ‘Un arreglo es una variable en la que se almacena, en forma secuencial, un conjunto de datos del mismo tipo a partir de una posición inicial’.

Como toda variable, un arreglo tiene las siguientes características:

Nombre= identificador único.

Tipo= ent, float, string, booleano, otra lista, todos los elementos de un arreglo deben ser del mismo tipo.

Dirección= El nombre del arreglo actúa como un puntero a esta dirección de memoria.

A : a1 a2 . . . ai−1 ai ai+1 . . . ak−1 ak , ejemplo texto autor Mancilla Herrera.

Contenido= Elementos del arreglo.

Y atributos como tamaño: Indica la cantidad total de elementos que el arreglo.

índice[ ] = Los índices generalmente comienzan desde 0 para el primer elemento, 1 para el segundo, y así sucesivamente.

- Como conclusión cito a Mancilla Herrera, A. (2015). “Sin embargo, el programador puede tener distintas vistas lógicas para la manipulación del mismo conjunto de datos”.
- En la programación y el diseño de algoritmos, está la forma lógica en que puedo visualizar y enfocar el problema, realizando conjuntos, siendo crítico, infiriendo el desarrollo, buscando relaciones, y otra forma es como organizo los datos y estos se almacenan realmente en la memoria de la computadora. Esto puede ser a través de arreglos contiguos, listas enlazadas, tablas hash, árboles, grafos, la estructura de los datos, los que una vez que codifico, debo saber cual es la mejor forma de acceder a ellos y lograr que el sistema funcione permitiendo ser legible por otros desarrolladores y lo más escalable posible.

```
# aprendiendo operaciones con matrices y numpy en python.
inventario_matriz = np.array([
    ["Samsung", "Neo QLED 8K", 75, 1000, "Quantum 4K", "Sí", None, 10, 3999.99],
    ["Apple", "MacBook Pro 16", 16.2, 1000, "M2 Pro", None, "16GB", 25, 2499.00],
    ["HP", "Spectre x360", 14, 500, "Intel i7", None, "16GB", 15, 1599.00],
    ["Lenovo", "ThinkPad X1 Carbon", 14, 1000, "Intel i7", None, "16GB", 30, 1899.00],
    ["Dell", "XPS 15", 15.6, 2000, "Intel i9", None, "32GB", 12, 2299.00],
    ["HP", "Pavilion Aero 13", 13.3, 256, "AMD Ryzen 7", None, "8GB", 40, 899.00],
    ["Apple", "MacBook Air M1", 13.3, 1000, "M1", None, "8GB", 20, 1299.00],
```

```
    ["Asus", "ROG Zephyrus G14", 14, 1000, "AMD Ryzen 9", None, "16GB", 18, 1799.00],
    ["Xiaomi", "Redmi Note 10", 6.43, 128, "Snapdragon 678", "48MP", "6GB", 50, 249.00],
])
laptops_1TB_mas = []

#for producto in inventario_matriz:
    #modelo = producto[MODELO]
    #almacenamiento = producto[ALMACENAMIENTO] para guardar modelo y almacenamiento en una variable y recorrer el arreglo sin numpy.
laptops_1TB_mas=inventario_matriz[inventario_matriz[:,ALMACENAMIENTO]>=1000]
print(laptops_1TB_mas)

precio_maximo = np.max(precios)
precio_minimo = np.min(precios)
print(f"Precio máximo: ${precio_maximo:.2f}")
print(f"Precio mínimo: ${precio_minimo:.2f}")
```

Ejecución consola.

```
[['Samsung' 'Neo QLED 8K' 75 1000 'Quantum 4K' 'Sí' None 10 3999.99]
 ['Apple' 'MacBook Pro 16' 16.2 1000 'M2 Pro' None '16GB' 25 2499.0]
 ['Lenovo' 'ThinkPad X1 Carbon' 14 1000 'Intel i7' None '16GB' 30 1899.0]
 ['Dell' 'XPS 15' 15.6 2000 'Intel i9' None '32GB' 12 2299.0]
 ['Apple' 'MacBook Air M1' 13.3 1000 'M1' None '8GB' 20 1299.0]
 ['Asus' 'ROG Zephyrus G14' 14 1000 'AMD Ryzen 9' None '16GB' 18 1799.0]]
Precio máximo: $1500.00
Precio mínimo: $1100.00
```

REFERENCIAS BIBLIOGRÁFICAS

Mancilla Herrera, A. (2015). Diseño y construcción de algoritmos: (ed.). Barranquilla, Colombia: Universidad del Norte. Recuperado de https://elibro.net/es/ereader/iacc/69931?page=245.

https://rise.articulate.com/share/ixm69TwMhyJXN3F_oQyXrmCtZ7Ik58-3#/